

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

ОТЧЁТ
о лабораторной работе № 1
«Код Грея. Мультимножества и операции над ними»
по дисциплине «Дискретная математика»

Выполнил
студент гр. 5130201/50302

_____ Р. Ю. Дужнов

Проверил
преподаватель

_____ А. В. Востров

«____» _____ 2026 г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Математическое описание	4
1.1 Множества	4
1.2 Мультимножества	4
1.3 Бинарный код Грея	5
1.4 Универсум в данной работе	6
2 Особенности реализации	7
2.1 Структуры данных	7
2.2 Генерация универсума	7
2.3 Заполнение мультимножеств	7
2.4 Вспомогательные функции	8
2.5 Операции над мультимножествами	8
2.6 Пользовательское меню и проверка ввода	9
3 Результаты работы	11
3.1 Сценарий 1: ручное заполнение, корректный ввод	11
3.2 Сценарий 2: автоматическое заполнение	15
3.3 Сценарий 3: некорректный ввод	19
3.4 Сценарий 4: пустое мультимножество	21
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26
ПРИЛОЖЕНИЕ А Текст программы	27

ВВЕДЕНИЕ

Реализована консольная программа, которая строит универсум мультимножеств из бинарного кода Грея разрядности n и выполняет операции над двумя мультимножествами [1]. Реализовано:

- а) генерация универсума из 2^n кодов Грея;
- б) ручное и автоматическое заполнение двух мультимножеств заданной мощности;
- в) теоретико-множественные операции: объединение, пересечение, разности $A \setminus B$ и $B \setminus A$, симметрическая разность, дополнения $\bar{A}$ и $\bar{B}$;
- г) арифметические операции: сумма, разность, произведение, деление;
- д) меню и проверка диапазонов вводимых значений.

Структуры данных: универсум — массив `int universe[2^n]`; мультимножество — массив пар «код Грея, кратность» `int multiset[m][2]`, где m не превышает мощности.

Стек: C++, g++ (MinGW), Windows 11.

1 Математическое описание

1.1 Множества

Множество — совокупность различных элементов [1]. Все множества рассматриваются как подмножества универсума U .

Операции над $A, B \subseteq U$ [1]:

$$A \cup B = \{x \mid x \in A \text{ или } x \in B\}, \quad (1)$$

$$A \cap B = \{x \mid x \in A \text{ и } x \in B\}, \quad (2)$$

$$A \setminus B = \{x \mid x \in A \text{ и } x \notin B\}, \quad (3)$$

$$A \triangle B = (A \setminus B) \cup (B \setminus A), \quad (4)$$

$$\bar{A} = U \setminus A, \quad (5)$$

где $A \cup B$ — объединение;

$A \cap B$ — пересечение;

$A \setminus B$ — разность;

$A \triangle B$ — симметрическая разность;

$\bar{A}$ — дополнение множества A до универсума U .

Тождество $A \setminus B = A \cap \bar{B}$ используется для определения разности мультимножеств.

1.2 Мультимножества

Мультимножество — совокупность элементов, допускающая повторения [1]. Мультимножество A над U задаётся функцией кратности $k_A: U \rightarrow \{0, 1, 2, \dots\}$ — числом вхождений x в A . Запись:

$$A = \{x_1^{k_A(x_1)}, x_2^{k_A(x_2)}, \dots\}, \quad (6)$$

где x_i — элементы универсума;

$k_A(x_i)$ — кратность элемента x_i в мультимножестве A .

Элементы с нулевой кратностью не записываются. Мощность — сумма кратностей:

$$|A| = \sum_{x \in U} k_A(x). \quad (7)$$

Универсум — мультимножество с кратностями $k_U(x)$; для любого A над

U выполняется $k_A(x) \leq k_U(x)$.

Кратность элемента $x \in U$ в результате операций.

Теоретико-множественные операции:

$$k_{A \cup B}(x) = \max(k_A(x), k_B(x)), \quad (8)$$

$$k_{A \cap B}(x) = \min(k_A(x), k_B(x)), \quad (9)$$

$$k_{\bar{A}}(x) = k_U(x) - k_A(x), \quad (10)$$

$$k_{A \setminus B}(x) = \min(k_A(x), k_U(x) - k_B(x)), \quad (11)$$

$$k_{A \triangle B}(x) = \max(k_{A \setminus B}(x), k_{B \setminus A}(x)), \quad (12)$$

где $k_U(x)$ — кратность элемента x в универсуме.

Формула (11) следует из $A \setminus B = A \cap \bar{B}$ и формул (9), (10); формула (12) — из определения (4). Вычисляются оба варианта разности ($A \setminus B, B \setminus A$) и дополнения ($\bar{A}, \bar{B}$).

Арифметические операции:

$$k_{A+B}(x) = k_A(x) + k_B(x), \quad (13)$$

$$k_{A-B}(x) = \max(k_A(x) - k_B(x), 0), \quad (14)$$

$$k_{A \cdot B}(x) = k_A(x) \cdot k_B(x), \quad (15)$$

$$k_{A/B}(x) = \left\lfloor \frac{k_A(x)}{k_B(x)} \right\rfloor, \quad k_B(x) > 0, \quad (16)$$

где $\lfloor \cdot \rfloor$ — целая часть числа.

Арифметическая разность (14), в отличие от (11), не зависит от универсума. При $k_B(x) = 0$ деление (16) не определено, элемент в результат не включается.

1.3 Бинарный код Грея

Код Грея разрядности n — последовательность всех 2^n двоичных наборов длины n , в которой соседние наборы различаются ровно в одном разряде [1].

Рекурсивное построение (отражённый код): G_1 — наборы 0, 1; далее

$$G_{n+1} = (0G_n, 1G_n^R), \quad (17)$$

где G_n — последовательность наборов кода Грея разрядности n ;
 G_n^R — та же последовательность в обратном порядке.

Наборы на стыке половин ($0g$ и $1g$) различаются одним разрядом.

В программе используется нерекурсивная формула:

$$g(i) = i \oplus \lfloor i/2 \rfloor, \quad (18)$$

где $g(i)$ — i -й набор кода Грея;

$\oplus$ — поразрядное сложение по модулю 2;

$\lfloor i/2 \rfloor$ — число i , сдвинутое на один разряд вправо.

Разряды кода: $g_j = b_j \oplus b_{j+1}$, где b_j — j -й разряд i . Код Грея для $n = 3$ — в таблице 1.

Таблица 1 — Бинарный код Грея разрядности 3

Номер i	Двоичная запись i	Код Грея $g(i)$	Десятичное значение $g(i)$
0	000	000	0
1	001	001	1
2	010	011	3
3	011	010	2
4	100	110	6
5	101	111	7
6	110	101	5
7	111	100	4

1.4 Универсум в данной работе

Универсум — все 2^n кодов Грея разрядности n . Кратность каждого элемента в универсуме:

$$k_U(x) = \max(|A|, |B|), \quad (19)$$

где $|A|$, $|B|$ — мощности мультимножеств, заданные пользователем.

Так как $k_A(x) \leq |A|$, выбор (19) гарантирует $k_A(x) \leq k_U(x)$ для обоих мультимножеств и неотрицательность дополнения (10).

2 Особенности реализации

2.1 Структуры данных

Файлы: `gray_code.cpp` — генерация кода Грея, заполнение, вывод и операции над мультимножествами; `lab1.cpp` — функция `main` и меню. Текст программы — в приложении А.

Структуры данных:

- универсум — `int universe[size], size = 2n; universe[i]` хранит $g(i)$;
- мультимножество — `int multiset[m][2]`, строка — пара «код Грея, кратность», m не превышает мощности;
- результат операции — `int res[size][2]` того же формата.

Число заполненных строк хранится отдельно (`count1`, `count2`, `resCount`). Код может повторяться в нескольких строках; его кратность равна сумме кратностей этих строк (`getMultiplicity`). Все массивы — массивы переменной длины (расширение `g++`), размещаются в стеке.

2.2 Генерация универсума

```
int conversionToGrayCode(int i)
```

Возвращает $i \wedge (i \gg 1)$ по формуле (18).

- вход: i — номер набора, $i \geq 0$;
- выход: $g(i)$; при $i < 0$ — сообщение об ошибке и завершение программы.

```
bool fillTheUniverse(int uni[], int size)
```

Записывает $g(i)$ в `uni[i]` для $i = 0, \dots, size-1$ и выводит универсум.

- вход: `uni` — массив универсума, $size = 2^n$;
- выход: заполненный `uni`; `true`.

2.3 Заполнение мультимножеств

```
int manualMethodOfFillingMultiset(int uni[][2], int power, int size)
```

Цикл до исчерпания мощности `power`: ввод номера $i \in [0, size)$ и кратности из $[1, \text{остаток мощности}]$; в очередную строку записывается пара $(g(i), \text{кратность})$.

- вход: `uni` — массив мультимножества, `power` — мощность,

size — размер универсума;

– выход: заполненный uni; число строк.

```
int automaticMethodOfFillingMultiset(int uni[][2], int power, int size)
```

То же, что ручное заполнение, но значения генерирует mt19937 (зерно от random_device): номер — равномерно из $[0, \text{size})$, кратность — из $[1, \text{остаток мощности}]$. Сумма кратностей равна power.

– вход и выход: как у manualMethodOfFillingMultiset.

2.4 Вспомогательные функции

```
int getMultiplicity(int ms[][2], int elementCount, int code)
```

Возвращает $k_A(x)$ — сумму кратностей строк с кодом code. Сложность $O(m)$, m — число строк.

– вход: ms, elementCount — мультимножество и число строк; code — код Грея;

– выход: кратность; 0, если код отсутствует.

```
void addElement(int res[][2], int &resCount, int code, int multiplicity)
```

Добавляет строку (code, multiplicity) в res, если multiplicity > 0.

– вход: res, resCount — результат и число строк; code, multiplicity — код и кратность;

– выход: дополненный res, resCount увеличен на 1.

```
void multisetOutput(int uni[][2], int elementCount)
```

Выводит строки «номер. Gray code: код; Multiplicity: кратность» или сообщение о пустом мультимножестве.

2.5 Операции над мультимножествами

Схема всех функций операций: для каждого $\text{uni}[i]$ кратности в операндах вычисляет getMultiplicity, кратность результата — формула операции, запись — addElement. Возвращается число строк результата. Параметры:

- a, aCount — первое мультимножество и число его строк;
- b, bCount — второе мультимножество и число его строк;
- uni, size — универсум и число его элементов;
- universeMultiplicity — $k_U(x)$ по формуле (19);

– res — массив результата.

Соответствие функций и формул — в таблице 2.

Таблица 2 — Функции операций над мультимножествами

Функция	Операция	Формула
multisetUnion	Объединение $A \cup B$	(8)
multisetIntersection	Пересечение $A \cap B$	(9)
multisetComplement	Дополнение $\bar{A}$	(10)
multisetDifference	Разность $A \setminus B$	(11)
multisetSymmetricDifference	Симметрическая разность $A \triangle B$	(12)
multisetArithmeticSum	Арифметическая сумма $A + B$	(13)
multisetArithmeticDifference	Арифметическая разность $A - B$	(14)
multisetArithmeticProduct	Арифметическое произведение $A \cdot B$	(15)
multisetArithmeticDivision	Арифметическое деление A/B	(16)

Параметр `universeMultiplicity` принимают `multisetComplement` (один операнд), `multisetDifference` и `multisetSymmetricDifference`. $B \setminus A$ и $\bar{B}$ вычисляются теми же функциями с переставленными аргументами.

```
void allOperationsOutput(...)
```

Вызывает функции всех 11 операций и выводит результаты.

- вход: оба мультимножества с числом строк, универсум, `size`, k_U ;
- выход: результаты на экране.

Сложность операции: 2^n итераций по универсуму, в каждой — `getMultiplicity` по обоим операндам; итого $O(2^n \cdot (m_A + m_B))$, где m_A , m_B — числа строк мультимножеств.

2.6 Пользовательское меню и проверка ввода

Порядок ввода в `main`:

- разрядность n от 0 до 30;
- для каждого мультимножества — способ заполнения (1 — вручную, 2 — автоматически) и мощность ≥ 0 ;
- при ручном заполнении — номера и кратности элементов.

Каждое значение запрашивается в цикле до попадания в допустимый диапазон (таблица 3).

Таблица 3 — Проверки пользовательского ввода

Вводимое значение	Допустимый диапазон	Реакция на ошибку
Разрядность n	$0 \leq n \leq 30$	Сообщение и повторный ввод
Пункт меню	1 или 2	Повторный вывод меню
Мощность мультимножества	≥ 0	Повторный запрос
Номер элемента	$0 \leq i < 2^n$	Сообщение и повторный ввод
Кратность элемента	от 1 до оставшейся мощности	Сообщение и повторный ввод

Ограничение кратности остатком мощности гарантирует, что сумма кратностей равна заданной мощности.

3 Результаты работы

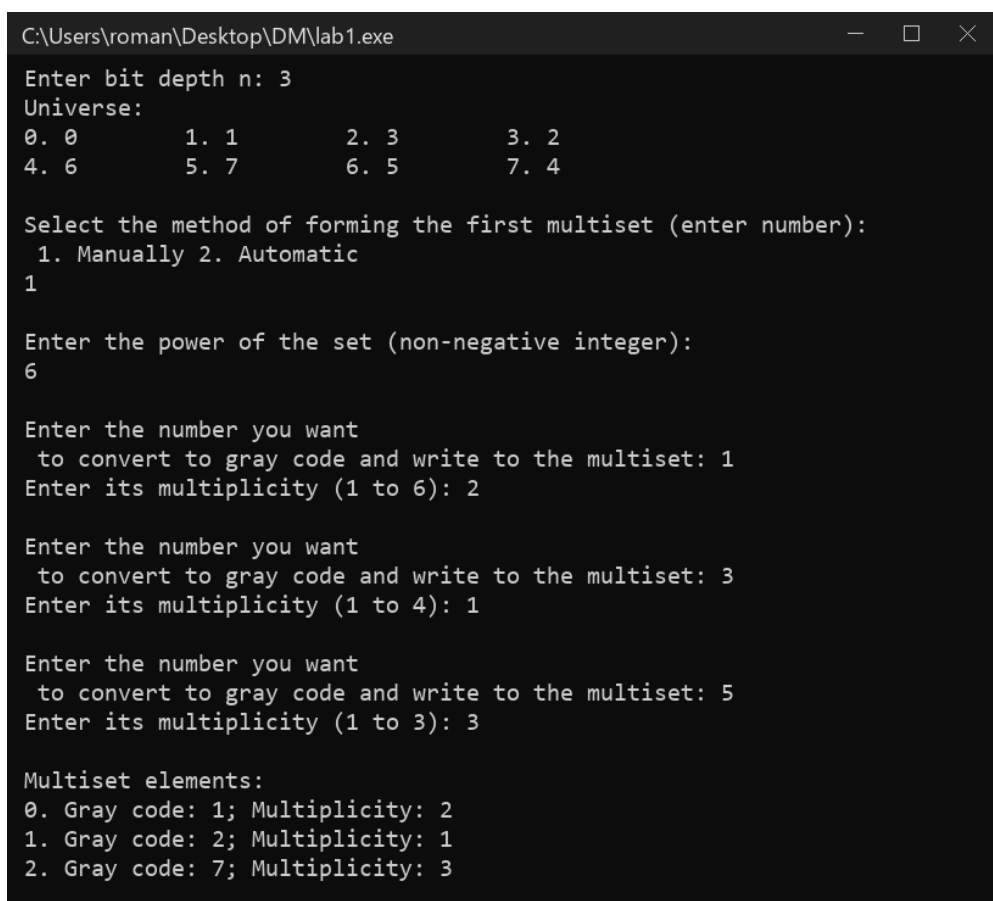
Программа проверена на четырёх сценариях; на рисунках — вывод консоли.

3.1 Сценарий 1: ручное заполнение, корректный ввод

$n = 3$, ручное заполнение. A : мощность 6, номера 1, 3, 5 с кратностями 2, 1, 3. B : мощность 5, номера 3, 5, 6 с кратностями 2, 1, 2. По таблице 1:

$$A = \{1^2, 2^1, 7^3\}, \quad B = \{2^2, 7^1, 5^2\}, \quad k_U(x) = \max(6, 5) = 6.$$

Ввод — рисунки 1, 2; результаты — рисунки 3, 4.



```
C:\Users\roman\Desktop\DM\lab1.exe
Enter bit depth n: 3
Universe:
0. 0      1. 1      2. 3      3. 2
4. 6      5. 7      6. 5      7. 4

Select the method of forming the first multiset (enter number):
1. Manually 2. Automatic
1

Enter the power of the set (non-negative integer):
6

Enter the number you want
to convert to gray code and write to the multiset: 1
Enter its multiplicity (1 to 6): 2

Enter the number you want
to convert to gray code and write to the multiset: 3
Enter its multiplicity (1 to 4): 1

Enter the number you want
to convert to gray code and write to the multiset: 5
Enter its multiplicity (1 to 3): 3

Multiset elements:
0. Gray code: 1; Multiplicity: 2
1. Gray code: 2; Multiplicity: 1
2. Gray code: 7; Multiplicity: 3
```

Рисунок 1 — Сценарий 1: универсум и ручное заполнение первого мультимножества

```
C:\Users\roman\Desktop\DM\lab1.exe

Select the method of forming the second multiset (enter number):
1. Manually 2. Automatic
1

Enter the power of the set (non-negative integer):
5

Enter the number you want
to convert to gray code and write to the multiset: 3
Enter its multiplicity (1 to 5): 2

Enter the number you want
to convert to gray code and write to the multiset: 5
Enter its multiplicity (1 to 3): 1

Enter the number you want
to convert to gray code and write to the multiset: 6
Enter its multiplicity (1 to 2): 2

Multiset elements:
0. Gray code: 2; Multiplicity: 2
1. Gray code: 7; Multiplicity: 1
2. Gray code: 5; Multiplicity: 2
```

Рисунок 2 — Сценарий 1: ручное заполнение второго мультимножества

```
C:\Users\roman\Desktop\DM\lab1.exe
Set-theoretic operations
(Universe multiplicity of each element: 6)

Union A U B
Multiset elements:
0. Gray code: 1; Multiplicity: 2
1. Gray code: 2; Multiplicity: 2
2. Gray code: 7; Multiplicity: 3
3. Gray code: 5; Multiplicity: 2

Intersection A ^ B
Multiset elements:
0. Gray code: 2; Multiplicity: 1
1. Gray code: 7; Multiplicity: 1

Difference A \ B
Multiset elements:
0. Gray code: 1; Multiplicity: 2
1. Gray code: 2; Multiplicity: 1
2. Gray code: 7; Multiplicity: 3

Difference B \ A
Multiset elements:
0. Gray code: 2; Multiplicity: 2
1. Gray code: 7; Multiplicity: 1
2. Gray code: 5; Multiplicity: 2

Symmetric difference A (+) B
Multiset elements:
0. Gray code: 1; Multiplicity: 2
1. Gray code: 2; Multiplicity: 2
2. Gray code: 7; Multiplicity: 3
3. Gray code: 5; Multiplicity: 2
```

Рисунок 3 — Сценарий 1: объединение, пересечение, разности и симметрическая разность

```
C:\Users\roman\Desktop\DM\lab1.exe
Complement not(A)
Multiset elements:
0. Gray code: 0; Multiplicity: 6
1. Gray code: 1; Multiplicity: 4
2. Gray code: 3; Multiplicity: 6
3. Gray code: 2; Multiplicity: 5
4. Gray code: 6; Multiplicity: 6
5. Gray code: 7; Multiplicity: 3
6. Gray code: 5; Multiplicity: 6
7. Gray code: 4; Multiplicity: 6

Complement not(B)
Multiset elements:
0. Gray code: 0; Multiplicity: 6
1. Gray code: 1; Multiplicity: 6
2. Gray code: 3; Multiplicity: 6
3. Gray code: 2; Multiplicity: 4
4. Gray code: 6; Multiplicity: 6
5. Gray code: 7; Multiplicity: 5
6. Gray code: 5; Multiplicity: 4
7. Gray code: 4; Multiplicity: 6

Arithmetic operations

Arithmetic sum A + B
Multiset elements:
0. Gray code: 1; Multiplicity: 2
1. Gray code: 2; Multiplicity: 3
2. Gray code: 7; Multiplicity: 4
3. Gray code: 5; Multiplicity: 2

Arithmetic difference A - B
Multiset elements:
0. Gray code: 1; Multiplicity: 2
1. Gray code: 7; Multiplicity: 2

Arithmetic product A * B
Multiset elements:
0. Gray code: 2; Multiplicity: 2
1. Gray code: 7; Multiplicity: 3

Arithmetic division A / B
Multiset elements:
0. Gray code: 7; Multiplicity: 3
```

Рисунок 4 — Сценарий 1: дополнения и арифметические операции

Ручной расчёт по формулам (8)–(16) (таблицы 4, 5) совпадает с выводом программы. Проверк: $k_B(x) = 0$, элемент в результат деления не входит.

Таблица 4 — Проверка теоретико-множественных операций (сценарий 1, $k_U = 6$)

Код x	k_A	k_B	$A \cup B$	$A \cap B$	$A \setminus B$	$B \setminus A$	$A \triangle B$	$\overline{A}$	$\overline{B}$
1	2	0	2	0	2	0	2	4	6
2	1	2	2	1	1	2	2	5	4
7	3	1	3	1	3	1	3	3	5
5	0	2	2	0	0	2	2	6	4
0, 3, 6, 4	0	0	0	0	0	0	0	6	6

Таблица 5 — Проверка арифметических операций (сценарий 1)

Код x	k_A	k_B	$A + B$	$A - B$	$A \cdot B$	A/B
1	2	0	2	2	0	—
2	1	2	3	0	2	0
7	3	1	4	2	3	3
5	0	2	2	0	0	0
0, 3, 6, 4	0	0	0	0	0	—

Различие разностей для элемента 7: теоретико-множественная — $\min(3, 6-1) = 3$ (ограничение дополнением $\overline{B}$), арифметическая — $3-1 = 2$.

3.2 Сценарий 2: автоматическое заполнение

$n = 4$, автоматическое заполнение, мощности 7 и 5. Получено $A = \{12^6, 7^1\}$, $B = \{12^4, 7^1\}$, $k_U = 7$ (рисунки 5–8).

```
C:\Users\roman\Desktop\DM\lab1.exe
Enter bit depth n: 4
Universe:
0. 0      1. 1      2. 3      3. 2
4. 6      5. 7      6. 5      7. 4
8. 12     9. 13    10. 15    11. 14
12. 10    13. 11    14. 9     15. 8

Select the method of forming the first multiset (enter number):
1. Manually 2. Automatic
2

Enter the power of the set (non-negative integer):
7

Multiset elements:
0. Gray code: 12; Multiplicity: 6
1. Gray code: 7; Multiplicity: 1

Select the method of forming the second multiset (enter number):
1. Manually 2. Automatic
2

Enter the power of the set (non-negative integer):
5

Multiset elements:
0. Gray code: 12; Multiplicity: 4
1. Gray code: 7; Multiplicity: 1
```

Рисунок 5 — Сценарий 2: универсум разрядности 4 и автоматическое заполнение мультимножеств

```
C:\Users\roman\Desktop\DM\lab1.exe
Set-theoretic operations
(Universe multiplicity of each element: 7)

Union A U B
Multiset elements:
0. Gray code: 7; Multiplicity: 1
1. Gray code: 12; Multiplicity: 6

Intersection A ^ B
Multiset elements:
0. Gray code: 7; Multiplicity: 1
1. Gray code: 12; Multiplicity: 4

Difference A \ B
Multiset elements:
0. Gray code: 7; Multiplicity: 1
1. Gray code: 12; Multiplicity: 3

Difference B \ A
Multiset elements:
0. Gray code: 7; Multiplicity: 1
1. Gray code: 12; Multiplicity: 1

Symmetric difference A (+) B
Multiset elements:
0. Gray code: 7; Multiplicity: 1
1. Gray code: 12; Multiplicity: 3
```

Рисунок 6 — Сценарий 2: объединение, пересечение, разности и симметрическая разность

```
C:\Users\roman\Desktop\DM\lab1.exe
Complement not(A)
Multiset elements:
0. Gray code: 0; Multiplicity: 7
1. Gray code: 1; Multiplicity: 7
2. Gray code: 3; Multiplicity: 7
3. Gray code: 2; Multiplicity: 7
4. Gray code: 6; Multiplicity: 7
5. Gray code: 7; Multiplicity: 6
6. Gray code: 5; Multiplicity: 7
7. Gray code: 4; Multiplicity: 7
8. Gray code: 12; Multiplicity: 1
9. Gray code: 13; Multiplicity: 7
10. Gray code: 15; Multiplicity: 7
11. Gray code: 14; Multiplicity: 7
12. Gray code: 10; Multiplicity: 7
13. Gray code: 11; Multiplicity: 7
14. Gray code: 9; Multiplicity: 7
15. Gray code: 8; Multiplicity: 7

Complement not(B)
Multiset elements:
0. Gray code: 0; Multiplicity: 7
1. Gray code: 1; Multiplicity: 7
2. Gray code: 3; Multiplicity: 7
3. Gray code: 2; Multiplicity: 7
4. Gray code: 6; Multiplicity: 7
5. Gray code: 7; Multiplicity: 6
6. Gray code: 5; Multiplicity: 7
7. Gray code: 4; Multiplicity: 7
8. Gray code: 12; Multiplicity: 3
9. Gray code: 13; Multiplicity: 7
10. Gray code: 15; Multiplicity: 7
11. Gray code: 14; Multiplicity: 7
12. Gray code: 10; Multiplicity: 7
13. Gray code: 11; Multiplicity: 7
14. Gray code: 9; Multiplicity: 7
15. Gray code: 8; Multiplicity: 7
```

Рисунок 7 — Сценарий 2: дополнения

```
C:\Users\roman\Desktop\DM\lab1.exe
Arithmetic operations

Arithmetic sum A + B
Multiset elements:
0. Gray code: 7; Multiplicity: 2
1. Gray code: 12; Multiplicity: 10

Arithmetic difference A - B
Multiset elements:
0. Gray code: 12; Multiplicity: 2

Arithmetic product A * B
Multiset elements:
0. Gray code: 7; Multiplicity: 1
1. Gray code: 12; Multiplicity: 24

Arithmetic division A / B
Multiset elements:
0. Gray code: 7; Multiplicity: 1
1. Gray code: 12; Multiplicity: 1
```

Рисунок 8 — Сценарий 2: арифметические операции

Суммы кратностей равны мощностям: $6+1 = 7$, $4+1 = 5$. Для элемента 12: $A \setminus B = \min(6, 7 - 4) = 3$, $B \setminus A = \min(4, 7 - 6) = 1$ — по формуле (11). Кратности суммы и произведения (10 и 24) превышают k_U : формулы (13), (15) универсумом не ограничены.

3.3 Сценарий 3: некорректный ввод

Проверка диапазонов (таблица 3). Введены:

- разрядность —3, 35, затем 2;
- пункты меню 0, 5, затем 1;
- мощность —4, затем 3;
- номер 9, —1 (диапазон $[0, 4)$), затем 2;
- кратность 5, 0 (диапазон $[1, 3]$), затем 3.

B : мощность 4, автоматическое заполнение (рисунки 9, 10).

```
C:\Users\roman\Desktop\DM\lab1.exe
Enter bit depth n: -3
Bit depth must be between 0 and 30. Try again: 35
Bit depth must be between 0 and 30. Try again: 2
Universe:
0. 0      1. 1      2. 3      3. 2

Select the method of forming the first multiset (enter number):
1. Manually 2. Automatic
0

Select the method of forming the first multiset (enter number):
1. Manually 2. Automatic
5

Select the method of forming the first multiset (enter number):
1. Manually 2. Automatic
1

Enter the power of the set (non-negative integer):
-4

Enter the power of the set (non-negative integer):
3

Enter the number you want
to convert to gray code and write to the multiset: 9

The number must be between 0 inclusive and 4 not inclusive!
Try again: -1

The number must be between 0 inclusive and 4 not inclusive!
Try again: 2
Enter its multiplicity (1 to 3): 5

The multiplicity must be between 1 and 3
Try again: 0

The multiplicity must be between 1 and 3
Try again: 3

Multiset elements:
0. Gray code: 3; Multiplicity: 3

Select the method of forming the second multiset (enter number):
1. Manually 2. Automatic
2

Enter the power of the set (non-negative integer):
4

Multiset elements:
0. Gray code: 3; Multiplicity: 3
1. Gray code: 1; Multiplicity: 1
```

Рисунок 9 — Сценарий 3: обработка некорректного ввода

```

C:\Users\roman\Desktop\DM\lab1.exe
Set-theoretic operations
(Universe multiplicity of each element: 4)

Union A U B
Multiset elements:
0. Gray code: 1; Multiplicity: 1
1. Gray code: 3; Multiplicity: 3

Intersection A ^ B
Multiset elements:
0. Gray code: 3; Multiplicity: 3

Difference A \ B
Multiset elements:
0. Gray code: 3; Multiplicity: 1

Difference B \ A
Multiset elements:
0. Gray code: 1; Multiplicity: 1
1. Gray code: 3; Multiplicity: 1

Symmetric difference A (+) B
Multiset elements:
0. Gray code: 1; Multiplicity: 1
1. Gray code: 3; Multiplicity: 1

Complement not(A)
Multiset elements:
0. Gray code: 0; Multiplicity: 4
1. Gray code: 1; Multiplicity: 4
2. Gray code: 3; Multiplicity: 1
3. Gray code: 2; Multiplicity: 4

Complement not(B)
Multiset elements:
0. Gray code: 0; Multiplicity: 4
1. Gray code: 1; Multiplicity: 3
2. Gray code: 3; Multiplicity: 1
3. Gray code: 2; Multiplicity: 4

Arithmetic operations

Arithmetic sum A + B
Multiset elements:
0. Gray code: 1; Multiplicity: 1
1. Gray code: 3; Multiplicity: 6

Arithmetic difference A - B
Multiset elements:
Multiset is empty.

Arithmetic product A * B
Multiset elements:
0. Gray code: 3; Multiplicity: 9

Arithmetic division A / B
Multiset elements:
0. Gray code: 3; Multiplicity: 1

```

Рисунок 10 — Сценарий 3: результаты операций после корректного ввода

Каждое недопустимое значение отклонено с повторным запросом; в мультимножество не записано. Итог: $A = \{3^3\}$, $3 = g(2)$.

3.4 Сценарий 4: пустое мультимножество

$n = 2$; A мощности 0; $B = \{3^3\}$ (автоматически); $k_U = 3$ (рисунки 11, 12).

```
C:\Users\roman\Desktop\DM\lab1.exe
Enter bit depth n: 2
Universe:
0. 0      1. 1      2. 3      3. 2

Select the method of forming the first multiset (enter number):
1. Manually 2. Automatic
1

Enter the power of the set (non-negative integer):
0

Multiset elements:
Multiset is empty.

Select the method of forming the second multiset (enter number):
1. Manually 2. Automatic
2

Enter the power of the set (non-negative integer):
3

Multiset elements:
0. Gray code: 3; Multiplicity: 3

Set-theoretic operations
(Universe multiplicity of each element: 3)

Union A U B
Multiset elements:
0. Gray code: 3; Multiplicity: 3

Intersection A ^ B
Multiset elements:
Multiset is empty.

Difference A \ B
Multiset elements:
Multiset is empty.

Difference B \ A
Multiset elements:
0. Gray code: 3; Multiplicity: 3

Symmetric difference A (+) B
Multiset elements:
0. Gray code: 3; Multiplicity: 3
```

Рисунок 11 — Сценарий 4: пустое первое мультимножество и теоретико-множественные операции

```
C:\Users\roman\Desktop\DM\lab1.exe
Complement not(A)
Multiset elements:
0. Gray code: 0; Multiplicity: 3
1. Gray code: 1; Multiplicity: 3
2. Gray code: 3; Multiplicity: 3
3. Gray code: 2; Multiplicity: 3

Complement not(B)
Multiset elements:
0. Gray code: 0; Multiplicity: 3
1. Gray code: 1; Multiplicity: 3
2. Gray code: 2; Multiplicity: 3

Arithmetic operations

Arithmetic sum A + B
Multiset elements:
0. Gray code: 3; Multiplicity: 3

Arithmetic difference A - B
Multiset elements:
Multiset is empty.

Arithmetic product A * B
Multiset elements:
Multiset is empty.

Arithmetic division A / B
Multiset elements:
Multiset is empty.
```

Рисунок 12 — Сценарий 4: дополнения и арифметические операции

Результаты: $A \cup B = B$, $A \cap B = \emptyset$, $A \setminus B = \emptyset$, $B \setminus A = B$, $\overline{A} = U$. Элемент 3 отсутствует в $\overline{B}$: $3 - 3 = 0$. Деление даёт пустое мультимножество: $\lfloor 0/3 \rfloor = 0$.

ЗАКЛЮЧЕНИЕ

Реализовано:

- а) генерация универсума из 2^n кодов Грея по формуле $g(i) = i \oplus \lfloor i/2 \rfloor$;
- б) ручное и автоматическое заполнение двух мультимножеств заданной мощности;
- в) 11 операций: объединение, пересечение, две разности, симметрическая разность, два дополнения, арифметические сумма, разность, произведение, деление;
- г) проверка диапазонов ввода.

Корректность подтверждена четырьмя сценариями (ручной и автоматический ввод, некорректный ввод, пустое мультимножество) и ручным расчётом.

Достоинства:

- генерация кода Грея — $O(1)$ на элемент, без рекурсии;
- единая схема операций через функцию кратности: каждая функция прямо соответствует формуле;
- память под мультимножество зависит от мощности, а не от размера универсума;
- повторные записи одного элемента учитываются суммированием кратностей.

Недостатки:

- нет проверки формата ввода: нечисловое значение переводит `cin` в состояние ошибки, программа за циклируется;
- массивы переменной длины в стеке: при $n \geq 18$ или большой мощности — аварийное завершение из-за переполнения стека, хотя допускается $n \leq 30$;
- $k_U = \max(|A|, |B|)$ не задаётся пользователем; при большом k_U разность $A \setminus B$ вырождается в A (сценарий 1);
- повторные записи элемента при выводе не объединяются;
- кратности суммы и произведения могут превышать k_U .

Масштабирование: память под универсум и массив результата — $O(2^n)$, время одной операции — $O(2^n \cdot (m_A + m_B))$; увеличение n на 1 удваивает размер универсума. Пути улучшения:

- динамическая память (`std::vector`) для больших n ;

- хранение мультимножества как массива кратностей по номеру элемента универсума: доступ к кратности $O(1)$, время операции $O(2^n)$;
- проверка `cin.fail()` с очисткой потока.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Новиков, Ф. А. Дискретная математика для программистов : учебник для вузов / Ф. А. Новиков. — 3-е изд. — Санкт-Петербург : Питер, 2009. — 384 с.

ПРИЛОЖЕНИЕ А

Код программы

Файл gray_code.cpp:

```
1 #include <iostream>
2 #include <cmath>
3 #include <cstdlib>
4 #include <random>
5
6 using namespace std;
7
8 class IncorrectNumber {};
9
10 // Переводит целое неотрицательное число в код Грея
11 int conversionToGrayCode(int i) {
12     try {
13         if (i < 0) {
14             throw IncorrectNumber();
15         }
16         else {
17             return (i ^ (i >> 1));
18         }
19     }
20     catch (IncorrectNumber) {
21         cout << "Number less than zero:" << i << endl;
22         exit(1);
23     }
24 }
25
26 // Выводит универсум
27 void universeOutput(int uni[], int size) {
28     cout << "Universe:\n";
29
30     if (size == 1) {
31         cout << "1." << uni[0];
32         cout << endl;
33     }
34     else if (size == 2) {
35         cout << "1." << uni[0] << "UUUUUUUU";
36         cout << "2." << uni[1];
37         cout << endl;
38     }
39     else {
40         int count = 0;
41         for (int i = 0; i < size / 4; i++) {
42             for (int j = 0; j < 4; j++) {
43                 cout << count << "." << uni[i * 4 + j] << "UUUUUUUU";
44                 count += 1;
```

```

45         }
46         cout << endl;
47     }
48 }
49 }
50
51 // Заполняет универсум
52 bool fillTheUniverse(int uni[], int size) {
53     for (int i = 0; i < size; i++) {
54         uni[i] = conversionToGrayCode(i);
55     }
56
57     universeOutput(uni, size);
58     return true;
59 }
60
61 // Выводит мультимножество
62 void multisetOutput(int uni[][2], int elementCount) {
63     cout << "\nMultiset elements:\n";
64     if (elementCount == 0) {
65         cout << "Multiset is empty.\n";
66         return;
67     }
68     for (int i = 0; i < elementCount; i++) {
69         cout << i << ". Gray code:" << uni[i][0] << "; Multiplicity:" << uni
70         [i][1] << endl;
71     }
72 }
73
74 // Ручное заполнение мультимножества
75 int manualMethodOfFillingMultiset(int uni[][2], int power, int size) {
76     int i = 0;
77     int remainingPower = power;
78
79     while (remainingPower > 0) {
80         int number = 0;
81         cout << "\nEnter the number you want to convert to gray code and
82         write to the multiset:";
83         cin >> number;
84
85         while (number < 0 || number >= size) {
86             cout << "\nThe number must be between 0 inclusive and " << size <<
87             "\nnot inclusive!" << endl;
88             cout << "Try again:";
89             cin >> number;
90         }
91
92         int count = 0;

```

```

90         cout << "Enter its multiplicity (1 to " << remainingPower << "): ";
91         cin >> count;
92
93         while (count < 1 || count > remainingPower) {
94             cout << "\nThe multiplicity must be between 1 and " <<
remainingPower << endl;
95             cout << "Try again: ";
96             cin >> count;
97         }
98
99         uni[i][0] = conversionToGrayCode(number);
100        uni[i][1] = count;
101
102        remainingPower -= count;
103        i += 1;
104    }
105
106    multisetOutput(uni, i);
107    return i;
108 }
109
110 // Автоматическое заполнение мультимножества
111 int automaticMethodOfFillingMultiset(int uni[][2], int power, int size) {
112     random_device rd;
113     mt19937 gen(rd());
114
115     int i = 0;
116     int remainingPower = power;
117
118     while (remainingPower > 0) {
119         uniform_int_distribution<int> distNumber(0, size - 1);
120         int number = distNumber(gen);
121
122         uniform_int_distribution<int> distCount(1, remainingPower);
123         int count = distCount(gen);
124
125         uni[i][0] = conversionToGrayCode(number);
126         uni[i][1] = count;
127
128         remainingPower -= count;
129         i += 1;
130     }
131
132     multisetOutput(uni, i);
133     return i;
134 }
135
136

```

```

137 // Реализация всех функций
138
139
140 // Возвращает кратность элемента code в мультимножестве
141 // если( элемент записан несколько раз, кратности суммируются)
142 int getMultiplicity(int ms[][2], int elementCount, int code) {
143     int multiplicity = 0;
144     for (int i = 0; i < elementCount; i++) {
145         if (ms[i][0] == code) {
146             multiplicity += ms[i][1];
147         }
148     }
149     return multiplicity;
150 }
151
152 // Добавляет элемент в результат, если его кратность больше нуля
153 void addElement(int res[][2], int &resCount, int code, int multiplicity) {
154     if (multiplicity > 0) {
155         res[resCount][0] = code;
156         res[resCount][1] = multiplicity;
157         resCount += 1;
158     }
159 }
160
161
162 // ----- Теоретикомножественные- операции -----
163
164 // Объединение: max(a, b)
165 int multisetUnion(int a[][2], int aCount, int b[][2], int bCount, int uni[],
166     int size, int res[][2]) {
167     int resCount = 0;
168     for (int i = 0; i < size; i++) {
169         int ka = getMultiplicity(a, aCount, uni[i]);
170         int kb = getMultiplicity(b, bCount, uni[i]);
171         addElement(res, resCount, uni[i], max(ka, kb));
172     }
173     return resCount;
174 }
175
176 // Пересечение: min(a, b)
177 int multisetIntersection(int a[][2], int aCount, int b[][2], int bCount, int
178     uni[], int size, int res[][2]) {
179     int resCount = 0;
180     for (int i = 0; i < size; i++) {
181         int ka = getMultiplicity(a, aCount, uni[i]);
182         int kb = getMultiplicity(b, bCount, uni[i]);
183         addElement(res, resCount, uni[i], min(ka, kb));
184     }

```

```

183     return resCount;
184 }
185
186 // Дополнение:  $U - a$ 
187 int multisetComplement(int a[][2], int aCount, int uni[], int size, int
    universeMultiplicity, int res[][2]) {
188     int resCount = 0;
189     for (int i = 0; i < size; i++) {
190         int ka = getMultiplicity(a, aCount, uni[i]);
191         addElement(res, resCount, uni[i], universeMultiplicity - ka);
192     }
193     return resCount;
194 }
195
196 // Разность  $A \setminus B = A \cap \text{not}(B): \min(a, U - b)$ 
197 int multisetDifference(int a[][2], int aCount, int b[][2], int bCount, int uni
    [], int size, int universeMultiplicity, int res[][2]) {
198     int resCount = 0;
199     for (int i = 0; i < size; i++) {
200         int ka = getMultiplicity(a, aCount, uni[i]);
201         int kb = getMultiplicity(b, bCount, uni[i]);
202         addElement(res, resCount, uni[i], min(ka, universeMultiplicity - kb));
203     }
204     return resCount;
205 }
206
207 // Симметрическая разность  $(A \setminus B) \cup (B \setminus A): \max(\min(a, U - b), \min(b, U - a)$ 
     $)$ 
208 int multisetSymmetricDifference(int a[][2], int aCount, int b[][2], int bCount
    , int uni[], int size, int universeMultiplicity, int res[][2]) {
209     int resCount = 0;
210     for (int i = 0; i < size; i++) {
211         int ka = getMultiplicity(a, aCount, uni[i]);
212         int kb = getMultiplicity(b, bCount, uni[i]);
213         int aWithoutB = min(ka, universeMultiplicity - kb);
214         int bWithoutA = min(kb, universeMultiplicity - ka);
215         addElement(res, resCount, uni[i], max(aWithoutB, bWithoutA));
216     }
217     return resCount;
218 }
219
220
221 // ----- Арифметические операции -----
222
223 // Арифметическая сумма:  $a + b$ 
224 int multisetArithmeticSum(int a[][2], int aCount, int b[][2], int bCount, int
    uni[], int size, int res[][2]) {
225     int resCount = 0;

```

```

226     for (int i = 0; i < size; i++) {
227         int ka = getMultiplicity(a, aCount, uni[i]);
228         int kb = getMultiplicity(b, bCount, uni[i]);
229         addElement(res, resCount, uni[i], ka + kb);
230     }
231     return resCount;
232 }
233
234 // Арифметическая разность:  $a - b$ , если  $a > b$ , иначе 0
235 int multisetArithmeticDifference(int a[][2], int aCount, int b[][2], int
    bCount, int uni[], int size, int res[][2]) {
236     int resCount = 0;
237     for (int i = 0; i < size; i++) {
238         int ka = getMultiplicity(a, aCount, uni[i]);
239         int kb = getMultiplicity(b, bCount, uni[i]);
240         addElement(res, resCount, uni[i], max(ka - kb, 0));
241     }
242     return resCount;
243 }
244
245 // Арифметическое произведение:  $a * b$ 
246 int multisetArithmeticProduct(int a[][2], int aCount, int b[][2], int bCount,
    int uni[], int size, int res[][2]) {
247     int resCount = 0;
248     for (int i = 0; i < size; i++) {
249         int ka = getMultiplicity(a, aCount, uni[i]);
250         int kb = getMultiplicity(b, bCount, uni[i]);
251         addElement(res, resCount, uni[i], ka * kb);
252     }
253     return resCount;
254 }
255
256 // Арифметическое деление:  $a / b$  целая (часть), если  $b > 0$ , иначе 0
257 int multisetArithmeticDivision(int a[][2], int aCount, int b[][2], int bCount,
    int uni[], int size, int res[][2]) {
258     int resCount = 0;
259     for (int i = 0; i < size; i++) {
260         int ka = getMultiplicity(a, aCount, uni[i]);
261         int kb = getMultiplicity(b, bCount, uni[i]);
262         if (kb > 0) {
263             addElement(res, resCount, uni[i], ka / kb);
264         }
265     }
266     return resCount;
267 }
268
269
270 // Вызывает все операции и выводит их результаты

```

```

271 void allOperationsOutput(int a[][2], int aCount, int b[][2], int bCount, int
    uni[], int size, int universeMultiplicity) {
272     int res[size][2];
273     int resCount = 0;
274
275     cout << "\n\nSet-theoretic operations\n";
276     cout << "(Universe multiplicity of each element: " << universeMultiplicity
        << ")\n";
277
278     cout << "\nUnion  $A \cup B$ ";
279     resCount = multisetUnion(a, aCount, b, bCount, uni, size, res);
280     multisetOutput(res, resCount);
281
282     cout << "\nIntersection  $A \cap B$ ";
283     resCount = multisetIntersection(a, aCount, b, bCount, uni, size, res);
284     multisetOutput(res, resCount);
285
286     cout << "\nDifference  $A \setminus B$ ";
287     resCount = multisetDifference(a, aCount, b, bCount, uni, size,
        universeMultiplicity, res);
288     multisetOutput(res, resCount);
289
290     cout << "\nDifference  $B \setminus A$ ";
291     resCount = multisetDifference(b, bCount, a, aCount, uni, size,
        universeMultiplicity, res);
292     multisetOutput(res, resCount);
293
294     cout << "\nSymmetric difference  $A \oplus B$ ";
295     resCount = multisetSymmetricDifference(a, aCount, b, bCount, uni, size,
        universeMultiplicity, res);
296     multisetOutput(res, resCount);
297
298     cout << "\nComplement  $\text{not}(A)$ ";
299     resCount = multisetComplement(a, aCount, uni, size, universeMultiplicity,
        res);
300     multisetOutput(res, resCount);
301
302     cout << "\nComplement  $\text{not}(B)$ ";
303     resCount = multisetComplement(b, bCount, uni, size, universeMultiplicity,
        res);
304     multisetOutput(res, resCount);
305
306     cout << "\n\nArithmetic operations\n";
307
308     cout << "\nArithmetic sum  $A + B$ ";
309     resCount = multisetArithmeticSum(a, aCount, b, bCount, uni, size, res);
310     multisetOutput(res, resCount);
311

```

```

312     cout << "\nArithmetic_difference_A-B";
313     resCount = multisetArithmeticDifference(a, aCount, b, bCount, uni, size,
res);
314     multisetOutput(res, resCount);
315
316     cout << "\nArithmetic_product_A*B";
317     resCount = multisetArithmeticProduct(a, aCount, b, bCount, uni, size, res)
;
318     multisetOutput(res, resCount);
319
320     cout << "\nArithmetic_division_A/B";
321     resCount = multisetArithmeticDivision(a, aCount, b, bCount, uni, size, res
);
322     multisetOutput(res, resCount);
323 }

```

Файл lab1.cpp:

```

1  #include <iostream>
2  #include "gray_code.cpp"
3
4  using namespace std;
5
6  int main()
7  {
8      int n, size;
9      cout << "Enter_bit_depth_n:";
10     cin >> n;
11
12     while (n < 0 || n > 30) {
13         cout << "Bit_depth_must_be_between_0_and_30.Try_again:";
14         cin >> n;
15     }
16
17     size = 1 << n;
18     int universe[size] = {};
19
20     fillTheUniverse(universe, size);
21
22     // Формат записи: [[GrayCode, multiplicity], ...]
23
24     // Первое мультимножество
25     int choice1;
26
27     do {
28         cout << "\n\nSelect_the_method_of_forming_the_first_multiset_(enter_
number):\n1_Manually\n2_Automatic" << endl;
29         cin >> choice1;
30     }

```

```

31 while (choice1 != 1 && choice1 != 2);
32
33 int power1;
34 do {
35     cout << "\n\nEnter the power of the set (non-negative integer): " <<
endl;
36     cin >> power1;
37 }
38 while (power1 < 0);
39
40 int multiset1[power1 == 0 ? 1 : power1][2] = {};
41 int count1 = 0;
42
43 if (choice1 == 1) {
44     count1 = manualMethodOfFillingMultiset(multiset1, power1, size);
45 }
46 else {
47     count1 = automaticMethodOfFillingMultiset(multiset1, power1, size);
48 }
49
50 // Второе мультимножество
51 int choice2;
52
53 do {
54     cout << "\n\nSelect the method of forming the second multiset (enter
number): \n1. Manually \n2. Automatic" << endl;
55     cin >> choice2;
56 }
57 while (choice2 != 1 && choice2 != 2);
58
59 int power2;
60 do {
61     cout << "\n\nEnter the power of the set (non-negative integer): " <<
endl;
62     cin >> power2;
63 }
64 while (power2 < 0);
65
66 int multiset2[power2 == 0 ? 1 : power2][2] = {};
67 int count2 = 0;
68
69 if (choice2 == 1) {
70     count2 = manualMethodOfFillingMultiset(multiset2, power2, size);
71 }
72 else {
73     count2 = automaticMethodOfFillingMultiset(multiset2, power2, size);
74 }
75

```

```
76 // Кратность каждого элемента в универсуме: ни одна кратность
77 // в  $A$  или  $B$  не может превысить мощность своего мультимножества
78 int universeMultiplicity = max(power1, power2);
79
80 allOperationsOutput(multiset1, count1, multiset2, count2, universe, size,
universeMultiplicity);
81
82 return 0;
83 }
```